

1. Projektbeschreibung

Die BankApp ist ein datenbankgestütztes Backend-System für eine fiktive Bank. Das System verwaltet Kunden (Clients), ihre Konten, Bankkarten, Überweisungen und Sparzinskontoen.

Es gibt zwei Rollen:

- **Client (user):** Kann sich anmelden, seine eigenen Konten und Karten einsehen und Überweisungen an andere Karten tätigen.
- **Bank Manager:** Kann alles verwalten — Clients anlegen/bearbeiten/löschen, Karten ausgeben, Transaktionen einsehen und Sparzinskontoen verwalten. Nur der Manager darf neue Clients registrieren.

Das Frontend wird als WPF-Anwendung (C#, MVVM) gebaut und kommuniziert mit dem Backend über HTTP-Requests. Das Backend wird mit Python + FastAPI + SQLite umgesetzt und ist über Swagger UI ([/docs](#)) testbar.

2. Softwarevoraussetzungen

Entwicklung

Software	Version
Python	≥ 3.11
pip	≥ 23
Git	beliebig

Python-Pakete

Paket	Version (getestet)	Zweck
fastapi	0.115.x	Web-Framework
uvicorn	0.30.x	ASGI-Server
sqlalchemy	2.0.x	ORM / Datenbankbindung
pydantic[email]	2.x	Request- & Response-Validierung
python-jose[cryptography]	3.3.x	JWT-Erstellung und -Prüfung
passlib[bcrypt]	1.7.x	Passwort-Hashing (bcrypt)
fastapi-restful	0.6.x	Class-Based Views (@cbv)
python-multipart	0.0.x	Form-Daten (OAuth2-Login)

Installation

```
pip install fastapi uvicorn sqlalchemy pydantic[email] python-jose[cryptography] passlib[bcrypt] fastapi-restful python-multipart
```

Datenbank

Komponente	Version
SQLite	≥ 3.35 (in Python-Standardinstallation enthalten)

3. Domäne & Tabellen

Das System besteht aus **6 Tabellen**:

Tabelle	Beschreibung
accounts	Alle Benutzer (Clients + Manager) mit Rolle, Email, Passwort-Hash, Adresse, Geburtsdatum
cards	Bankkarten eines Accounts (IBAN, CardNr, Ablaufdatum, CVC, Status, Guthaben in Cent)
transactions	Überweisungen zwischen zwei Karten (Betrag in Cent, Datum, Beschreibung, Status)
interests	Sparzinskonten — einer Karte zugeordnet (Betrag, Zinssatz, ausgezahlt?)
bank	Bankdaten (Name, aktueller Zinssatz)
audit_logs	Protokollierung aller relevanten Aktionen nach Account

Kardinalitäten

Typ	Beziehung	Beispiel
1:n	Account → Cards	Ein Client hat mehrere Karten
1:n	Card → Interests	Eine Karte kann mehrere Sparzinskonten haben
m:n	Cards ↔ Cards via Transactions	Eine Karte sendet an viele, empfängt von vielen

Die $m:n$ -Beziehung wird durch `transactions` realisiert: eine Karte sendet an viele Karten (`from_id`) und empfängt von vielen Karten (`to_id`). Diese wird als eigene Zwischentabelle mit zwei Fremdschlüsseln umgesetzt.

4. Relationales Modell

accounts (<u>id</u>, email, password, firstname, lastname, phonenumber, address, birthdate, role, created_at)

cards (<u>id</u>, owner_id, iban, card_nr, expire_date, cvc, status, cents, created_at)

transactions (<u>id</u>, from_id, to_id, amount_cents, description, status, created_at)

interests (<u>id</u>, card_id, amount, interest_rate, withdrawn, created_at)

bank (<u>id</u>, bankname, interest_rate)

audit_logs (<u>id</u>, account_id, action, tablename, details, timestamp)

5. Normalformen

accounts

- **1NF**: Alle Attribute sind atomar (keine Listen, keine Wiederholungsgruppen).
- **2NF**: Kein zusammengesetzter Primärschlüssel → 2NF automatisch erfüllt.
- **3NF**: Alle Nicht-Schlüssel-Attribute hängen direkt vom Primärschlüssel `id` ab. Keine transitiven Abhängigkeiten.

cards

- **1NF**: Alle Attribute atomar. Jede Karte hat genau eine IBAN, eine card_nr usw.

- **2NF:** Einfacher PK → automatisch erfüllt.
- **3NF:** `owner_id` ist FK, alle anderen Attribute beschreiben direkt die Karte selbst, nicht den Account.

transactions

- **1NF:** Atomar. `from_id` und `to_id` sind zwei separate FK-Felder.
- **2NF:** Einfacher PK → automatisch erfüllt.
- **3NF:** Betrag, Datum und Status beschreiben die Transaktion, nicht die Karten. Keine transitiven Abhängigkeiten.

interests

- **1NF:** Atomar. Zinssatz und Betrag sind einzelne Felder.
- **2NF:** Einfacher PK → automatisch erfüllt.
- **3NF:** `amount` und `interest_rate` hängen direkt von `id` ab, nicht von `card_id`.

bank

- **1NF:** Atomar. Bankname und Zinssatz sind einzelne Felder.
- **2NF:** Einfacher PK → automatisch erfüllt.
- **3NF:** Alle Attribute beschreiben direkt die Bank. Keine transitiven Abhängigkeiten.

audit_logs

- **1NF:** Atomar. Jeder Eintrag beschreibt genau eine Aktion zu einem Zeitpunkt.
- **2NF:** Einfacher PK → automatisch erfüllt.
- **3NF:** `action`, `tablename`, `details` beschreiben den Log-Eintrag direkt. `account_id` ist nur FK-Referenz.

6. REST-API Endpunkte

Stack: Python · FastAPI · uvicorn · SQLite · Pydantic

Authentifizierung: JWT Bearer Token (OAuth2). Login über `/accounts/login` → gibt `access_token` zurück. Token wird im Header als `Authorization: Bearer <token>` mitgeschickt. Fehlender oder ungültiger Token → `401 Unauthorized`.

Vollständige interaktive Dokumentation mit Wertebereichen und Beispielen unter <http://127.0.0.1:8000/docs> (Swagger UI).

6.1 Accounts

Methode	Pfad	Beschreibung	Rolle	Codes
201, 409, 422				
200, 401				
GET	<code>/accounts/me</code>	Eigenen Account abrufen	alle	200, 401
GET	<code>/accounts</code>	Alle Accounts auflisten	manager	200
GET	<code>/accounts/{id}</code>	Einzelnen Account abrufen	alle	200, 404
PUT	<code>/accounts/me</code>	Eigenes Profil aktualisieren (Rolle kann nicht geändert werden)	alle	200, 422
PUT	<code>/accounts/{id}</code>	Account vollständig aktualisieren	manager	200, 404, 422

DELETE	/accounts/{id}	Account löschen	manager	200, 400, 404
--------	----------------	-----------------	---------	---------------

6.2 Cards – Karten CRUD

Methode	Pfad	Beschreibung	Rolle	Codes
GET	/cards	Eigene Karten abrufen	alle	200
GET	/cards/all	Alle Karten abrufen	manager	200
GET	/cards/{account_id}	Karten eines bestimmten Accounts	manager	200
POST	/cards	Neue Karte erstellen	manager	201, 404
PUT	/cards/{card_id}	Karte bearbeiten (Status, Ablaufdatum)	manager	200, 404
DELETE	/cards/{card_id}	Karte löschen	manager	200, 404

6.3 Transactions – Überweisungen

Methode	Pfad	Beschreibung	Rolle	Codes
GET	/transactions	Eigene Transaktionen (ein- und ausgehend)	alle	200
GET	/transactions/all	Alle Transaktionen	manager	200
GET	/transactions/account/{id}	Transaktionen eines bestimmten Accounts	manager	200
POST	/transactions	Überweisung erstellen (Betrag > 0, Guthaben prüfen)	alle	200, 400, 403, 404

6.4 Stats – Aggregation

Methode	Pfad	Beschreibung	Rolle	Codes
GET	/stats/transactions-per-account	COUNT + SUM Transaktionen pro Account (GROUP BY, JOIN)	manager	200

6.5 Interests – Sparzinskonten

Methode	Pfad	Beschreibung	Rolle	Codes
GET	/interests	Eigene Sparzinskonten	alle	200
GET	/interests/all	Alle Sparzinskonten	manager	200
POST	/interests	Sparkonto eröffnen (Betrag wird sofort abgebucht)	alle	200, 400, 403, 404
POST	/interests/{id}/withdraw	Zinsen auszahlen (Zinseszins-Formel)	alle	200, 400, 403, 404

6.6 Banks

Methode	Pfad	Beschreibung	Rolle	Codes
200, 404				
POST	/banks	Bank anlegen	manager	200
PUT	/banks/{bank_id}	Bank aktualisieren (Name, Zinssatz)	manager	200, 404

6.7 Audit Logs

Methode	Pfad	Beschreibung	Rolle	Codes
GET	/audit-logs	Alle Audit-Log-Einträge (neueste zuerst)	manager	200
GET	/audit-logs/{account_id}	Logs eines bestimmten Accounts	manager	200

7. Rollen & Authentifizierung

Rolle	Wert	Erlaubte Operationen
client	0	GET eigene Accounts/Karten/Transaktionen/Interessen, POST Überweisung, POST Sparkonto
manager	1	Alles — vollständiger CRUD auf allen Ressourcen

Umsetzung via **JWT Bearer Token**. Token-Erstellung beim Login, Ablauf nach 30 Minuten. Passwörter werden mit `bcrypt` gehasht (via `passlib`). Rolle wird im Token-Payload mitgespeichert und bei jedem Request geprüft.

8. Wertebereiche (wichtigste Felder)

Feld	Typ	Wertebereich
firstname, lastname	string	3–30 Zeichen, nur Buchstaben, Bindestriche, Leerzeichen
email	string	Gültige E-Mail-Adresse, eindeutig im System
password	string	≥ 8 Zeichen, mind. 1 Großbuchstabe, mind. 1 Ziffer
phonenumber	string	7–15 Zeichen, optional <code>+</code> am Anfang
address	string	5–100 Zeichen
birthdate	date	Format <code>YYYY-MM-DD</code>
role	int	<code>0</code> = client, <code>1</code> = manager
cents	int	≥ 0 (Kartenguthaben in Cent)
amount_cents	int	> 0 (Überweisungsbetrag in Cent)
iban	string	15–34 Zeichen, beginnt mit 2-buchst. Ländercode
card_nr	string	13–16 Ziffern
cvc	int	100–9999 (3- oder 4-stellig)
interest_rate	float	$0 < \text{Wert} \leq 1$ (z. B. <code>0.035</code> = 3,5 % p.a.)
amount (Sparkonto)	int	> 0 (Anlagebetrag in Cent)

9. Optionale Features (Nice-to-Have)

Feature	Beschreibung	Status
Erweiterte Filterung	<code>GET /accounts?name=Max</code> — Suche nach Name	offen
Pagination	<code>limit</code> und <code>offset</code> auf allen Listen-Endpunkten	offen
Erweiterte Aggregation	<code>GET /stats/balance-per-account</code> — Durchschnittsguthaben (AVG)	offen

10. Ordnerstruktur

```
BankAppAPI/
├── init_db.py           # Datenbank initialisieren (Tabellen anlegen)
├──
├── src/
│   ├── main.py         # App-Start, Logging, Middleware, Router einbinden
│   └── database.py     # SQLite-Verbindung, get_db()-Funktion
```

```
|   ├── models.py           # SQLAlchemy ORM-Modelle
|   ├── schemas.py         # Pydantic Request- & Response-Schemas (mit Field-
|   Constraints)
|   ├── auth.py            # JWT-Erstellung, Passwort-Hashing, Rollen-
|   ├── audit.py           # AuditLogger-Klasse
|   |
|   └── routers/
|       ├── accounts.py     # CRUD Accounts
|       ├── cards.py        # CRUD Karten
|       ├── transactions.py # Überweisungen
|       ├── interests.py    # Sparzinskonten
|       ├── bank.py         # Bankdaten
|       ├── auditlogs.py    # Audit-Log-Abfragen
|       └── stats.py        # Aggregationsendpunkte
|
└── doc/
    ├── DBI Projekt - BankApp.md
    └── erm.drawio
```

11. Bedienungsanleitung

Datenbank initialisieren

```
python init_db.py
```

API starten

```
uvicorn src.main:app --reload
```

Swagger UI erreichbar unter: <http://127.0.0.1:8000/docs>

Beispiel-Requests

1. Manager registrieren (POST /accounts/register)

```
{
  "firstname": "Max",
  "lastname": "Mustermann",
  "email": "max@bank.at",
  "password": "Sicher1!",
  "phonenummer": "+43664123456",
  "address": "Musterstraße 1, 1010 Wien",
  "birthdate": "1990-01-01",
  "role": 1
}
→ 201 Created
```

2. Login (POST /accounts/login)

```
Content-Type: application/x-www-form-urlencoded
```

```
username=max@bank.at&password=Sicher1!
```

```
→ 200 OK { "access_token": "eyJ...", "token_type": "bearer" }
```

3. Überweisung (POST /transactions)

```
Authorization: Bearer <token>
```

```
{  
  "iban_from": "AT611904300234573201",  
  "iban_to": "AT4832000000012345864",  
  "amount_cents": 5000,  
  "description": "Miete März"  
}
```

```
→ 200 OK
```

4. Sparkonto eröffnen (POST /interests)

```
Authorization: Bearer <token>
```

```
{  
  "card_id": 1,  
  "amount": 10000  
}
```

```
→ 200 OK { "id": 1, "amount": 10000, "interest_rate": 0.035, "withdrawn":  
false, ... }
```

5. Statistik (GET /stats/transactions-per-account)

```
Authorization: Bearer <token> (manager)
```

```
→ 200 OK
```

```
[  
  { "account_id": 1, "firstname": "Max", "lastname": "Mustermann",  
    "transaction_count": 3, "total_cents": 15000 }  
]
```

12. Mögliche Probleme und ihre Lösung

Problem 1: Route-Konflikt /cards/all vs. /cards/{account_id}

Problem: FastAPI verarbeitet Routen in Definitionsreihenfolge. Wenn `/cards/{account_id}` vor `/cards/all` definiert wird, wird `all` als `account_id` interpretiert.

Lösung: `/cards/all` vor `/cards/{account_id}` in der Klasse definieren.

Problem 2: SQLite CURRENT_TIMESTAMP vs. PostgreSQL now()

Problem: SQLAlchemy-Default `func.now()` erzeugt in SQLite ungültige Zeitstempel (liefert leere Strings).

Lösung: In den Modellen `server_default="CURRENT_TIMESTAMP"` verwenden, das SQLite nativ versteht.

Problem 3: AuditLogger erzeugt doppelte Einträge

Problem: Der AuditLogger wurde vor `db.commit()` mit einem separaten `db.commit()` aufgerufen, was die Transaktion vorzeitig abschloss und doppelte Log-Einträge erzeugte.

Lösung: `db.flush()` nach dem Haupt-Objekt verwenden (weist ID zu), dann AuditLogger aufrufen, dann ein einziges `db.commit()` am Ende.

Problem 4: 307-Redirect löscht Authorization-Header

Problem: FastAPI gibt bei Routen ohne abschließenden Slash (`/cards/all`) einen 307-Redirect zurück, wenn die Anfrage mit Slash kommt (`/cards/all/`). Der .NET `HttpClient` entfernt den `Authorization`-Header beim Folgen von Redirects → Backend antwortet mit 401.

Lösung: Im Frontend alle API-Aufrufe ohne trailing Slash formulieren. FastAPI-Routen werden immer ohne abschließenden Slash definiert.

Problem 5: JWT-Token läuft nach 5 Minuten ab (zu kurz)

Problem: Bei längeren Arbeitssessions mit dem Frontend wurde der Token während der Nutzung ungültig und löste einen Logout aus.

Lösung: Token-Ablaufzeit in `auth.py` auf 30 Minuten erhöht.

13. Must-Have vs. Nice-to-Have

Must-Have

- Login für Client und Manager (JWT)
- Client CRUD (Manager)
- Karten CRUD (Manager)
- Eigene Konten & Karten einsehen (Client)
- Überweisung erstellen (Client)
- Sparszinskonten verwalten (Eröffnen + Auszahlen mit Zinseszins)
- JOIN-Endpunkt (Transaktionen mit Kartendetails via `from_id/to_id`)
- Aggregationsendpunkt (Transaktionen pro Account, `GROUP BY + COUNT/SUM`)
- Logging in Datei (`api.log`) + Konsole via Middleware
- Audit-Log für alle schreibenden Aktionen
- Parametrisierte SQL-Abfragen (SQL-Injection-Schutz via SQLAlchemy ORM)

Nice-to-Have

- Filterparameter auf Listen-Endpunkten (`?name=...`)
- Pagination (`limit / offset`)
- Durchschnitts-Guthaben-Statistik (AVG)

14. Milestones

Datum	Ziel	Zuständigkeit
27.05.2026	RMs und ERMs	Chiara
28.05.2026	Modelle implementieren	Chiara
03.06.2026	Endpoints implementiert	Alexei
04.06.2026	Erste Demo	Chiara und Alexei
10.06.2026	Zusätzliche Endpoints implementieren	Alexei
11.06.2026	Refactoring	Alexei
17.06.2026	Backend funktioniert perfekt mit dem Frontend	Alexei
18.06.2026	Endpräsentation + Abgabe	Alexei & Chiara

15. Projekttagbuch

Datum	Was wurde gemacht	Wer
27.05.2026	ERM und RM erstellt, Domäne festgelegt	Chiara und Alexei
28.05.2026	SQLAlchemy-Modelle angelegt (accounts, cards, transactions, interests, audit_logs)	Alexei
28.05.2026	Datenbankverbindung (database.py) und Pydantic-Schemas (schemas.py) erstellt	Alexei
03.06.2026	JWT-Authentifizierung implementiert (auth.py)	Alexei
03.06.2026	Accounts-Router mit Register, Login, GET, DELETE	Alexei
03.06.2026	Cards-Router mit GET, POST, DELETE	Alexei
03.06.2026	Transactions-Router mit GET und POST (Überweisung mit Guthaben-Prüfung)	Alexei
03.06.2026	Interests-Router mit GET, POST, Withdraw-Endpoint (Zinseszins-Berechnung)	Alexei
04.06.2026	Erste Demo — API läuft, Swagger UI funktioniert	Alexei und Chiara
05.06.2026	AuditLogger implementiert, Bank-Router hinzugefügt	Alexei
10.06.2026	GET /accounts/{id} und PUT /accounts/{id} ergänzt	Alexei
10.06.2026	PUT /cards/{id} ergänzt, Route-Konflikt /cards/all gefixt	Alexei
10.06.2026	Aggregationsendpunkt GET /stats/transactions-per-account implementiert	Alexei
10.06.2026	Python-Logging (Konsole + api.log) via Middleware konfiguriert	Alexei
10.06.2026	AuditLogger in alle schreibenden Endpunkte integriert (Doppel-Commit-Bug gefixt)	Alexei
10.06.2026	HTTP-Statuscodes korrigiert (409 Conflict, 200 OK bei DELETE)	Alexei
11.06.2026	GET /transactions/account/{id} für Manager ergänzt	Alexei
11.06.2026	PUT /accounts/me (eigenes Profil, ohne Rollenwechsel) hinzugefügt	Alexei
12.06.2026	PUT /banks/{id} für Bankdaten-Update ergänzt	Alexei
13.06.2026	GET /audit-logs/{account_id} ergänzt	Alexei
16.06.2026	Swagger-Dokumentation vervollständigt (summary, description, Wertebereiche, Beispiele)	Alexei und Chiara
16.06.2026	Pydantic Field-Constraints in schemas.py ergänzt	Alexei und Chiara
17.06.2026	Backend-Frontend-Integration final getestet	Alexei
17.06.2026	Endpräsentation + Abgabe	Alexei & Chiara